Detailed Explanation

The Online Examination Network project explained step by step: concepts, how it works, how to read the evidence, and viva questions

1. The idea in plain words

Imagine a school exam where every student sits at a computer and all of them talk to one teacher's computer (the exam server). The roads between them are the network. If someone downloads a huge file on the same road, exam messages get stuck in the traffic jam. In a normal network nobody manages the road. In a Software-Defined Network (SDN) a central traffic controller watches every junction (switch) and can change the rules at any moment: send traffic another way, slow down troublemakers, or block a stranger. This project builds the exam, builds the road network, writes the traffic controller, and then measures how much the controller helps compared with an unmanaged network.

2. Key terms

Term	Meaning in this project
Socket / TCP	A socket is the endpoint a program uses to send data over a network. TCP delivers every byte, in order, so answers are never lost.
UDP	Fast delivery without guarantees. Used for the small events sent to the controller and for the background traffic that creates congestion.
SDN, controller	The switches only forward packets; the controller (Ryu) decides how. One central program sees the whole network.
OpenFlow	The language the controller uses to program switches (version 1.3).
Flow rule	A switch instruction: if a packet matches X, do Y. Each rule has a priority; the highest matching one wins.
Packet-In	When no rule matches, the switch asks the controller what to do with the packet.
LLDP	Small discovery packets the controller uses to find which switches are linked together.
Meter	An OpenFlow rate limiter: traffic above a set rate is dropped.
Mininet	Creates virtual hosts and switches on one computer so network experiments are repeatable.
Bottleneck	The deliberately slow 10 Mbps link between s1 and s4 that every exam packet must cross.
Baseline	The same network and the same exam but with a controller that does no management. It is the thing we compare against.
Percentile	The 95th percentile latency means 95% of requests were faster than this value; it exposes the slow tail that an average hides.
Jain's fairness index	(sum of x) squared divided by (n times sum of x squared), where x is each student's service rate (1 divided by average latency). 1.0 means everyone is served equally. Example: rates 1, 1, 1 give 1.0; rates 10, 1, 1 give 0.47.

3. How the exam works, step by step

Every message is a small JSON object preceded by 4 bytes that say how long it is. This matters because TCP is a stream of bytes with no marks between messages, so the receiver reads the length first and then exactly that many bytes.

Step	Message	What happens
1	LOGIN (ID, name)	The server checks the roster. Unknown ID: ERROR and a UNAUTHORIZED event to the controller. Duplicate login while connected: rejected.
2	LOGIN_OK, START	The student gets the number of questions and the time remaining. In web mode each student's 15 minutes start here.
3	GET_QUESTION, QUESTION	The client asks for question i and the server replies with the text and options. The client measures this round trip: that is the response time.
4	ANSWER, ACK	The answer is stored on the server (it can be changed until submission).

Step	Message	What happens
5	TIME_SYNC	The server periodically tells each student the time left. Only the server clock counts.
6	SUBMIT, RESULT	The server scores the answers and replies with the score (and a review in web mode).
7	(timer expires)	The server submits for the student automatically and sends RESULT with the auto-submitted flag.

Concurrency: the server starts one thread per connected student and protects shared data with locks. Each student also has a send lock, because both the student's own thread and the timer thread can write to the same connection. If a student's connection drops, reconnecting with the same ID resumes at the next unanswered question.

Web exam: a browser cannot open a raw TCP connection, so a small gateway program serves the page and opens one real TCP connection per student to the exam server. The exam server is unchanged, so the web students' traffic crosses the SDN just like the terminal clients' traffic.

4. How the controller works

Function	How it works
Topology discovery	Started with --observe-links, Ryu sends LLDP packets and learns every switch-to-switch link. The controller keeps these links in a graph. Until links are known it ignores traffic, so nothing is mis-learned.
Routing	On a Packet-In the controller learns where hosts are (only on host-facing ports), computes the shortest path through the graph, and installs a rule on every switch of that path. Rules expire after 30 seconds of silence. Exam traffic (TCP port 5000) gets its own higher-priority rules so it can be counted and protected separately.
Loop-free flooding	Broadcasts such as ARP are only sent along a spanning tree, so the backup path never creates a broadcast storm.
Monitoring	Every 2 seconds it reads the byte counters of every port and computes the Mbps. Values go to port_stats.csv and the decisions to sdn_events.csv.
Congestion response	If any port is above 8 Mbps for two polls in a row (about 4 seconds), the controller installs a meter on every switch and flushes the rules. Non-exam traffic now passes through the meter (2 Mbps per switch); exam traffic does not. The limit is removed only after at least 15 seconds, no meter drops, and load below the threshold twice in a row, so it does not flip on and off.
Link-failure recovery	When a link disappears, the controller deletes the learned rules. The next packet of each flow triggers a Packet-In and the shortest path is recomputed over what remains, which is now through s5. When the link returns the rules are flushed again so traffic goes back to the best path.
Blocking	The exam server sends a UDP message UNAUTHORIZED with the host's address. The controller installs a priority-300 drop rule for that address on all switches.
Baseline mode	Same forwarding, but none of: exam class, congestion limit, blocking, reroute after failure.

5. How the experiments work

tests/run_experiment.py builds the network, starts the exam server on h9, optionally starts iperf (a UDP flood from h10 to h12 across the bottleneck), starts N students spread over h1 to h8, optionally takes link s1-s4 down mid-exam, waits for the exam to end and saves everything under results/. Each student client logs the round-trip time of every request. tests/analyze.py turns those logs into mean, 95th-percentile and worst latency, completion rate and Jain's index. tests/run_matrix.py repeats this for every condition in both modes (28 runs), and tests/plot.py averages the repeats and draws the charts.

Why this is a fair comparison: both modes use the same topology, the same exam, the same flood and the same random answers logic; only the controller's behaviour differs.

6. Reading the evidence screenshots

The numbers refer to the figures in the Final Report with Evidence PDF.

Figure	What it shows	What to notice
1	Exam app test on the VM	Two students log in at the same instant (concurrency), EXAM START, scores, EXAM END.
2	Controller start	listener=udp/9999 and LINK_UP lines: topology discovery works.
3	First Mininet run	Students on 10.0.0.1 to 10.0.0.3 reach the server through the SDN switches.
4	Intruder blocked	APP_UNAUTHORIZED then BLOCKED 1 ms later: an application event changed the network.
5 and 6	Congestion, baseline versus SDN	Mean latency 1,822 ms versus 36 ms; worst request 24,574 ms versus 385 ms; fairness 0.77 versus 0.94.
7	Controller events during congestion	POLICE_ON at a peak of 14.1 Mbps with a 2,000 kbps limit. The load above 10 Mbps is possible because the switch counts what it offers to the link before the link's shaper drops the excess.
8 and 9	Link failure, SDN versus baseline	SDN: LINK_DOWN then REROUTE 4 ms later, worst request 207 ms. Baseline: 20 s outage, worst request 36,769 ms. Rows after APP_END are Mininet teardown noise.
10	Evaluation matrix	28 runs, 2 repeats each, and the averaged table the charts are drawn from.

7. Understanding the results

- **Why SDN wins under congestion:** the 12 Mbps flood exceeds the 10 Mbps link, so a queue builds and every packet waits, including exam packets. The controller notices the load, limits everything except exam traffic to 2 Mbps, and the queue drains. Exam latency falls back to near the idle level.
- **Why SDN wins on link failure:** the baseline keeps sending into the dead link until TCP's own retransmission timers recover after the link returns. The controller sees the failure within milliseconds and sends traffic over the backup path.
- **Why SDN is slightly slower with no load:** SDN mode installs separate rules for exam traffic, which may cost extra first-packet handling. This is a likely explanation that was not isolated by an experiment.
- **Why SDN fairness is 0.80 at 20 students under load:** likely the roughly 4 second detection delay in a short exam, so early requests suffer unevenly. The baseline value of 0.86 is also affected because 7.5% of its students did not finish and are missing from the calculation. This was not confirmed by examining per-student latencies.
- **How reliable are the numbers:** two repeats per condition. They show a very large effect, but they are indicative, not statistically tested.

8. Likely viva questions and answers

Q. What problem does the project solve?

It keeps an online exam usable when the network is congested or a link fails, by letting an SDN controller monitor and manage the exam traffic, and it measures the benefit against an unmanaged network.

Q. Why TCP for the exam?

Questions and answers must arrive complete and in order. TCP guarantees that; with UDP we would have to build retransmission ourselves.

Q. How do you find message boundaries in a TCP stream?

Each message has a 4-byte length prefix. The receiver reads the length and then exactly that many bytes (common/protocol.py).

Q. How does the server handle many students?

One thread per connection, locks around shared data, and a send lock per student because the timer thread and the student's thread can both send.

Q. Why is the timer on the server?

A student cannot gain time by changing a local clock. The server records the deadline, rejects late requests, and auto-submits.

Q. How does a browser use a TCP-based server?

A gateway serves the web page and opens one real TCP connection per student to the exam server, so the server and the SDN see ordinary TCP traffic.

Q. How does the controller learn the topology and avoid loops?

Ryu discovers links with LLDP (--observe-links). Floods use a spanning tree, so the backup path cannot create a broadcast storm.

Q. How does the controller detect and react to congestion?

It reads port byte counters every 2 seconds. Above 8 Mbps twice in a row it installs an OpenFlow meter that limits non-exam traffic to 2 Mbps per switch. It releases the limit after a hold time once the load is gone.

Q. Why not limit exam traffic too?

The goal is to protect it. Exam flows are a separate class with higher priority and no meter.

Q. What happens when a link fails?

The controller sees the link disappear, deletes the learned rules, and the next packets are routed through s5. When the link returns it flushes again so traffic goes back to the shortest path.

Q. How are the socket application and the SDN connected?

The exam server sends UDP events (START, LOGIN, UNAUTHORIZED, END) to the controller. UNAUTHORIZED makes the controller install a drop rule for that host.

Q. What is the baseline and why do you need it?

The same network with a controller that does no management. Without it we could not say the improvement comes from the SDN logic.

Q. Which metrics did you use?

Per-request mean, 95th-percentile and worst latency, completion rate, and Jain's fairness index over each student's service rate.

Q. Your SDN is slower when there is no load. Why?

Likely the extra handling of the separate exam flows on first packets. It is small compared with the gain under load, and we report it as a trade-off. We did not isolate the cause with an experiment.

Q. Fairness at 20 students is lower with SDN. Why?

Probably the detection delay of about 4 seconds in a short exam, and the baseline figure is distorted by students who never finished. We did not confirm this with per-student data.

Q. What would you improve?

Use the START event to protect exam traffic before congestion is detected, shorten the detection delay, add per-student rate fairness, and run more repeats with a statistical test.

Q. What if the controller crashes?

Rules already in the switches keep forwarding until they expire (30 seconds idle), but new conversations cannot be set up because nobody answers Packet-In.

Q. Why a separate Python 3.9 environment?

The Ryu version used does not run on the very new Python shipped with recent Ubuntu.

9. Numbers worth remembering

Item	Value
Topology	12 hosts, 4 switches (5 with backup path), 10 Mbps bottleneck s1-s4
Congestion trigger / limit	8 Mbps twice in a row / 2,000 kbps for non-exam traffic per switch
Flood used	12 Mbps UDP from h10 to h12
Congestion result (10 students)	Mean latency 1,822 ms baseline versus 36 ms SDN; fairness 0.77 versus 0.94
Matrix (2 repeats)	Mean latency 99x, 47x and 16x lower with SDN at 5, 10 and 20 students; baseline completion 92.5% at 20 students
Link failure (20 s outage)	Worst request 36,769 ms baseline versus 207 ms SDN; reroute 4 ms after LINK_DOWN
Weak points to admit	SDN slightly slower with no load; SDN fairness 0.80 at 20 students; only 2 repeats